



Tarea 03

No. de cuenta	Nombre
320148204	González Lucero Alan Uriel
321225087	Rivera Jiménez Isaac
321306102	Tapia Sánchez Jesús



Complejidad Computacional

Índice

1	Ejercicio 1	2
1.1	Respuestas	2
2	Ejercicio 2	7
2.1	Respuestas	7
2.2	Modelo	7
2.3	Cómputo	7
3	Ejercicio 3	9
3.1	Respuestas	9
4	Ejercicio 4	15
4.1	Solución al Inciso a	15
4.2	Solución al Inciso b	17
4.3	Solución al Inciso c	19
5	Ejercicio 5	20
5.1	Solución	20

Ejercicio 1

Enunciado

En este ejercicio vamos a asumir que una variante de la Máquina de Turing es equivalente a la versión estándar (vista en clase) si el conjunto de lenguajes L que puede ser decidido por el modelo de la variante es el mismo conjunto de lenguajes que puede ser decidido por la versión estándar. Justifica tu respuesta.

- Una MT con una cinta infinita en ambas direcciones que se puede mover arbitrariamente en cualquier dirección. Es decir, la función de transición es de la forma:

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \mathbb{Z}$$

donde el último entero especifica que tan lejos se mueve hacia la derecha (si es positivo) o a la izquierda (si es negativo).

- Una Máquina de Turing que utiliza el alfabeto infinito $\Gamma = \mathbb{N}$.
- Una Máquina que no puede moverse hacia la izquierda. Es decir, la función de transición es de la forma:

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{-, \rightarrow\}$$

Respuestas

Denotemos por $D(X)$ al conjunto de lenguajes que puede decidir el modelo X . La definición del enunciado establece que X y Y son equivalentes si:

$$D(X) = D(Y)$$

Como es una igualdad de conjuntos, la relación es reflexiva, simétrica y transitiva. Si dos variantes son equivalentes al modelo estándar, entonces son equivalentes entre sí.

Para demostrar que una variante es equivalente al modelo estándar hay que probar dos contenciones:

$$D(\text{estándar}) \subseteq D(\text{variante}) \quad \text{y} \quad D(\text{variante}) \subseteq D(\text{estándar})$$

Cada una de estas inclusiones se prueba mediante una simulación. Es decir, dada una máquina de un modelo, se construye una del otro modelo que acepta, rechaza o no se detiene exactamente cuando la original lo hace. Es decir que dos modelos son equivalentes si cada uno puede imitar al otro. No importa si uno es más rápido o más cómodo; solo nos interesa qué problemas pueden resolver. Si queremos demostrar que no son equivalentes, basta encontrar un solo lenguaje que un modelo decida y el otro no.

1

Para demostrar que esta variante no es más poderosa que el modelo estándar, mostramos cómo una MT estándar M puede simular paso a paso a la variante M' .

$$(D(\text{estándar}) \subseteq D(\text{variante}))$$

Si se supone una máquina de Turing estándar arbitraria $M = (Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$ con:

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{-1, 0, 1\}$$

Construimos la máquina variante $M' = (Q, \Sigma, \Gamma, \delta', q_0, q_a, q_r)$ con:

$$\delta'(q, a) = \delta(q, a)$$

Como $\{-1, 0, 1\} \subset \mathbb{Z}$, la máquina M' no necesita hacer ninguna adaptación. La entrada se coloca desde la celda 0 hacia la derecha o colocando un símbolo nuevo antes de empezar, tal que $* \notin \Gamma$. M' ejecuta exactamente las mismas transiciones que M . Si M entra en bucle, M' entra en el mismo bucle; si M se detiene en q_a o q_r , M' se hace exactamente lo mismo.

$$(D(\text{variante}) \subseteq D(\text{estándar}))$$

Lo hacemos en dos pasos: primero quitamos los saltos y luego quitamos la cinta biinfinita.

El estado interno de M incluye un bit auxiliar que indica en qué pista se encuentra virtualmente la cabeza de M' en ese instante (M^+ o M^-).

Se ejecución de un paso de cómputo de M' para simular un único paso de M dado por la regla $\delta'(q, a) = (q', b, k)$ con $k \in \mathbb{Z}$:

Primeramente M lee la celda actual en la pista activa, verifica el símbolo a , escribe b en la pista activa y cambia el estado interno hacia q' . Como la función de transición δ' es un conjunto finito de reglas, existe una constante $k_{max} = \max\{|k|\}$ para la máquina M' . M simula el movimiento de k posiciones mediante un bucle interno de $|k|$ movimientos de un solo paso ($d \in \{-1, 1\}$), desta manera tenemos el siguiente funcionamiento

- Si $k > 0$, M desplaza su cabeza un espacio a la derecha por cada unidad de k .
- Si la cabeza cruza la posición 0 en la cinta física hacia la izquierda mientras está en M^+ , M cambia su estado interno a M^- y comienza a mover su cabeza hacia la derecha en la cinta física para recorrer los índices negativos.
- Si $k < 0$, realiza el proceso inverso.
- Si M' acepta x tras un número finito de pasos de M' , la simulación en M alcanzará el estado q'_a y M aceptará.

- Si M' rechaza x : Tras un número finito de pasos de M' , la simulación en M alcanzará el estado q_r y M rechazará.
- Si M' entra en bucle infinito con x por lo tanto la máquina M continuará ejecutando los pasos de simulación indefinidamente, entrando también en un bucle infinito.

Dado que existe una construcción algorítmica explícita para simular cualquier máquina M' mediante una máquina M de ambos lados tanto para la variante como para el modelo estándar preservando exactamente su comportamiento para toda entrada, se demuestra que ambos modelos tienen exactamente el mismo poder de cómputo y son equivalentes.

2

Primera Suposición

Lo primero es notar que esto no es una máquina de Turing según la definición vista en clase, que exige que Γ sea finito. En cuanto $\Gamma = \mathbb{N}$, el dominio de δ es $Q \times \mathbb{N}$, que es infinito, por lo tanto la tabla de transición tiene infinitos renglones. El enunciado no dice cómo está dada esa tabla, y la respuesta depende de ello.

En lo que sigue suponemos que el alfabeto de entrada $\Sigma = \{\sigma_1, \dots, \sigma_k\}$ sigue siendo finito, con $\Sigma \subseteq \mathbb{N}$ y $\sqcup \in \mathbb{N} \setminus \Sigma$. Por ejemplo, $\sqcup = 0$ y $\sigma_d = d$.

Segunda suposición

Si suponemos que la función de transición δ tiene una descripción finita o computable, para que el modelo se mantenga o se considere un algoritmo ejecutable, se debe exigir que la función de transición δ pueda ser descrita de forma finita. Esto puede ocurrir de dos maneras.

Primero, si solo un número finito de números de $\mathbb{N}$ tienen reglas de transición explícitas y todos los demás números aplican un comportamiento uniforme (por ejemplo, mantener el símbolo y detenerse), la máquina solo puede leer o escribir un conjunto finito de símbolos a lo largo de cualquier ejecución. Es decir que en la ejecución se reduce a una MT estándar tomando los números utilizados a un alfabeto finito Γ .

Segundo, suponemos una máquina M' , tal que la máquina variante con alfabeto infinito $\Gamma' = \mathbb{N}$ y función de transición δ' computable. Puesto que δ' es computable, existe una Máquina de Turing estándar M_δ que, dada la entrada $\langle q, n \rangle$, se detiene en un número finito de pasos produciendo en su cinta la tupla $\langle q', n', d \rangle$, donde $d \in \{-1, 0, 1\}$.

Construimos una Máquina de Turing estándar M con alfabeto finito $\Gamma = \{0, 1, \#, \hat{\#}, _ \}$ que

representa el contenido de la cinta de M' guardando el valor numérico de cada celda codificado en binario y separado por el delimitador $\#$.

El delimitador previo a la celda que está escanendo la cabeza virtual de M' se identifica con el símbolo marcado $\hat{\#}$. Para simular un paso de cómputo de M' , la máquina estándar M realiza el siguiente algoritmo:

1. Copia a una zona de trabajo de la cinta el estado actual q junto con el número n (en binario) ubicado en el bloque marcado por $\hat{\#}$
2. Ejecuta a M_δ como subrutina con la entrada $\langle q, n \rangle$ para obtener el resultado $\delta'(q, n) = (q', n', d)$
3. Reescribe el bloque binario n con el nuevo valor n' , desplazando el resto de la cinta a la derecha o izquierda si n' ocupa más o menos dígitos que n .
4. Desplaza la marca $\hat{\#}$ al delimitador del bloque vecino según la dirección d . Si el movimiento llega a una zona no inicializada, M crea un nuevo bloque con la representación binaria del símbolo blanco
5. Si $q' = q'_{accept}$, M pasa a su estado q_{accept} y acepta; si $q' = q'_{reject}$, M pasa a su estado q_{reject} y rechaza.

Como M_δ siempre se detiene por ser una función computable, cada paso de M' se simula en un número finito de pasos de M . Por lo tanto, M preserva exactamente el comportamiento de M' (aceptación, rechazo o bucle infinito), demostrando que ambos modelos son equivalentes.

3

Analizamos los dos movimientos permitidos en $\{-, \rightarrow\}$:

Movimiento a la derecha ($\rightarrow$): La cabeza lectora avanza a la siguiente celda sin posibilidad de retornar. Una vez que la cabeza abandona una celda hacia la derecha, la información escrita a la izquierda queda inaccesible para el resto del cómputo.

Movimiento neutro ($-$): Permite a la máquina permanecer en la celda actual, modificar el símbolo y cambiar de estado. Sin embargo, dado que el número de estados $|Q|$ y el tamaño del alfabeto $|\Gamma|$ son conjuntos finitos, el número de configuraciones locales distintas en una sola celda está acotado por $|Q| \times |\Gamma|$. Si M' permanece en la misma celda por más de $|Q| \times |\Gamma|$ pasos, se repetirá una configuración, cayendo en un bucle infinito.

Por lo tanto, el movimiento neutro solo permite realizar una cantidad de modificaciones finita y acotada antes de avanzar obligatoriamente a la derecha o detenerse. Debido a esto, la cinta no puede usarse como memoria auxiliar reutilizable. M' procesa la entrada en una sola

pasada de izquierda a derecha, comportándose como un autómata finito determinista.

Demostración por contradicción: Para demostrar que dos modelos no son equivalentes, basta probar que existe al menos un lenguaje L decidible por el modelo estándar ($L \in D(M)$) que no puede ser decidido por el modelo variante ($L \notin D(M')$). Así que, por contradicción, supongamos, que existe una máquina variante M' que decide L utilizando un conjunto finito de estados Q' con $|Q'| = p$ estados.

Consideremos el conjunto de $p + 1$ prefijos formados únicamente por ceros:

$$\{\epsilon, 0, 0^2, \dots, 0^p\}$$

Al terminar de procesar cualquiera de estos prefijos 0^i (con $0 \leq i \leq p$), la máquina M' no puede haber aceptado ni rechazado todavía. Esto se debe a que la cadena $0^i 1^i \in L$ (debe ser aceptada) y la cadena $0^i 1^{i+1} \notin L$ (debe ser rechazada), y ambas comparten el mismo prefijo 0^i .

Como M' debe continuar el cómputo y posicionar su cabeza en la primera celda del sufijo restante, existen $p + 1$ historias de cómputo sobre la entrada de ceros, pero solo p estados posibles en Q' . Deben existir dos índices distintos $i \neq j$ (con $0 \leq i, j \leq p$) tales que M' alcanza la primera celda después del prefijo en el mismo estado interno $q \in Q'$.

Dado que la función de transición de M' no permite desplazamientos hacia la izquierda, M' no puede releer o modificar las celdas donde guardo los prefijos 0^i y 0^j . Por lo tanto, el comportamiento de M' depende únicamente de el estado actual q y la subcadena restante por leer a la derecha.

Si ahora proporcionamos el sufijo 1^i a ambas simulaciones, entonces para la entrada $0^i 1^i$, M' llega a la primera celda del 1^i en el estado q y posteriormente acepta (pues $0^i 1^i \in L$).

Para la entrada $0^j 1^i$, M' llega a la primera celda del 1^i exactamente en el mismo estado q . Como el estado inicial de esta fase y el sufijo restante son idénticos, M' ejecutará exactamente la misma secuencia de pasos y también aceptará. En este caso M' acepta la cadena $0^j 1^i$. Sin embargo, dado que $i \neq j$, la cadena $0^j 1^i \notin L$, por lo que M' debió haberla rechazado.

La suposición de que existía una máquina M' de la variante capaz de decidir L es falsa. Por lo tanto, $L \notin D(M')$, lo que demuestra que:

$$D(M') \subsetneq D(M)$$

Ejercicio 2

Enunciado

Investiga un modelo de cómputo (diferente a los mencionados en clase y en esta tarea) que sea equivalente a la máquina de Turing. Describe y ejemplifica un cómputo (ejecución de un algoritmo) en dicho modelo.

Respuestas

Modelo

El modelo seleccionado fue: *Redes de Petri*. **Definición:** Una red de Petri es un conjunto formado por:

$$R = \{P, T, Pre, Post\}$$

Donde:

- P es un conjunto de lugares de cardinal n .
- T es un conjunto de transiciones de cardinal m .
- Pre es la aplicación de incidencia previa definida como:

$$Pre : P \times T \rightarrow Naturales$$

- $Post$ la aplicación de incidencia posterior que viene definida como:

$$Post : P \times T \rightarrow Naturales$$

Entonces tenemos que la Red R es el mapa del sistema que define los lugares de los nodos, las acciones que pueden ocurrir y cómo se conectan las flechas. Por eso se requiere de lo siguiente:

Definición: Una red marcada, es un conjunto formado por $\{R, M\}$ donde:

- R es una Red de Petri.
- M es una aplicación denominada marcado y:

$$M : P \rightarrow Naturales$$

Añadiendo eso, ahora se tiene una red con datos y memoria en un momento concreto.

Cómputo

Buscador del Máximo.

Sea $V = [x_1, x_2, \dots, x_n]$ una lista de n números naturales. Sea $RN = (R, M_0)$ la red marcada y sea $R = (P, T, Pre, Post)$ la red de petri construida de la siguiente manera:

$P :$

$$P = \{p_1, p_2, \dots, p_n\} \cup \{p_{max}, p_{aux}\}$$

- $p_1, \dots, p_n$: Lugares de entrada que almacenarán los valores numéricos.
- p_{max} : Lugar donde se acumulará el valor máximo global al finalizar la ejecución.
- p_{aux} : Lugar auxiliar para la extracción y comparación temporal.

T :

$$T = \{t_{init}\} \cup \{t_{comp_i} | 2 \leq i \leq n\} \cup \{t_{clear}\}.$$

- t_{init} : Inicializa el candidato a máximo cargando las marcas de p_1 en p_{max} .
- t_{comp_i} : Compara activamente las fichas restantes del elemento x_i con las del máximo acumulado hasta el momento.
- t_{clear} : Descarta los residuos sobrantes del elemento comparado si resultaba ser menor o igual.

Pre y *Post*: Para t_{init} :

- $Pre(p_1, p_{init} = 1)$
- $Post(p_{max}, t_{init}) = 1.$

Para las comparaciones entre x_i y el candidato actual en p_{max} :

- $Pre(p_i, t_{comp_i}) = 1$
- $Pre(p_{max}, t_{comp_i}) = 1$
- $Post(p_{aux}, t_{comp_i}) = 1$

M_0 : El estado inicial del sistema se define como: $M_0 : P \rightarrow \mathbb{N}$:

$$M_0(p) = \begin{cases} x_i & \text{si } p = p_i \text{ para } i \in \{1, \dots, n\} \\ 0 & \text{si } p = p_i \text{ para } \{p_{max}, p_{aux}\} \end{cases}$$

Ahora, sea $V = [3, 5, 2]$ una lista de elementos: M_0 :

- $M_0(p_1) = 3.$
- $M_0(p_2) = 5.$
- $M(p_3) = 2.$
- $M_0(p_{max}) = 0, M_0(p_{aux}) = 0.$

Ahora t_{init} se dispara 3 veces consecutivas consumiendo las marcas de p_1 , teniendo ahora M_1

$$M_1(p_1) = 0, M_1(p_2) = 5, M_1(p_3) = 2, M_1(p_{max}) = 3$$

El máximo candidato actual es 3.

Ahora se dispara la transición de comparación t_{comp_2} restando 1 unidad de p_2 y de p_{max} de forma simultanea sucede:

1. t_{comp} se dispara 3 veces (el mínimo entre 5 y 3).
2. Quedan 0 fichas en p_{max} , 2 fichas restantes en p_2 y 3 fichas acumuladas en p_{aux} .
3. Como p_2 aún tiene fichas, significa que x_2 es mayor que el máximo anterior. Por lo que se mueven las 2 fichas restantes de p_2 y las 3 fichas de p_{aux} hacia p_{max} .

Ahora se tiene M_2 :

$$M_2(p_1) = 0, M_2(p_2) = 0, M_2(p_3) = 2, M_2(p_{max}) = 5.$$

El máximo candidato actual es 5.

Se dispara la transición de comparación t_{comp_3} entre p_3 y p_{max} :

1. t_{comp_3} se dispara 2 veces (el mínimo entre 2 y 5).
2. p_3 se queda con 0 fichas, p_{max} queda con 3 fichas y p_{aux} recibe 2 fichas.
3. Como p_3 no conservó fichas, significa que $x_3 \leq 5$. Las dos fichas de p_{aux} regresan a p_{max} para restaurar su valor original (3+2=5).

M_f :

$$M_f(p_1) = 0, M_f(p_2) = 0, M_f(p_3) = 0, M_f(p_{max}) = 5.$$

El cómputo se da por finalizado al no existir ninguna transición habilitada. El resultado se obtiene evaluando el marcado del lugar de salida p_{max}

Ejercicio 3

Enunciado

Prueba que la colección de lenguajes decidibles es cerrada bajo las siguientes operaciones:

- (a) Unión.
- (b) Intersección.
- (c) Concatenación.
- (d) Complemento.

¿Para cuáles de las operaciones anteriores se mantiene la cerradura para las clases P, NP ?

Respuestas

a

Para demostrar que $L_3(M_3) = L_1 \cup L_2$, suponemos lo siguiente.

Suponemos que L_1 es decidible si existe una máquina M_1 tal que $L(M_1) = L_1$ y M_1 es total. De la misma forma suponemos un L_2 que es decidible si existe una máquina M_2 tal que $L(M_2) = L_2$ y M_2 es total.

Por lo tanto lo que se quiere demostrar es que existe una M_3 tal que $L_3(M_3) = L_1 \cup L_2$ y M_3 es total.

Suponemos el funcionamiento de $M_3 = U$ como el siguiente:

1. U simula M_1 con x Si $M_1(x) = 1$, entonces U acepta
2. U simula M_2 con x Si $M_2(x) = 1$, entonces U acepta
3. U en otro caso rechaza

Si suponemos que $M_1(x) = 1$, entonces $L_1(M_1)$ acepta, dada la union, si x bajo la ejecucion de M_1 acepta, entonces $L_1 \cup L_2$ se cumple y acepta.

Si suponemos que $M_2(x) = 1$ acepta, entonces $L_2(M_2)$ acepta, dada la union, si x bajo la ejecucion de M_2 acepta, entonces $L_2 \cup L_1$ se cumple y acepta.

Si $M_1(x) = 0$, entonces $L_1(M_1)$ rechaza, dada la union, $M_2(x) = 1$, por lo tanto $L_1 \cup L_2$ se cumple. Sucede lo mismo para M_2

Por doble contencion tenemos que $L_3 = L_1 \cup L_2$.

Para analizar la complejidad temporal de M_3 con una entrada x de longitud $n = |x|$: Sea $t_1(n)$ el tiempo que tarda M_1 en procesar una entrada de longitud n . Sea $t_2(n)$ el tiempo que tarda M_2 en procesar una entrada de longitud n .

Como M_3 ejecuta M_1 y (en el peor de los casos) posteriormente M_2 en secuencia sobre la misma entrada x , el tiempo de ejecución total $T_{M_3}(n)$ está acotado por la suma de los tiempos de ambas máquinas:

$$T_{M_3}(n) = O(t_1(n) + t_2(n))$$

Caso Polinomial: Si $M_1 \in TIME(n^{k_1})$ y $M_2 \in TIME(n^{k_2})$ para constantes $k_1, k_2 \geq 1$:

$$T_{M_3}(n) = O(n^{k_1} + n^{k_2}) = O(n^{\max(k_1, k_2)})$$

Pero como solo se quiere saber si es decidible o no, no es necesario analizar su complejidad temporal, pues solo debemos garantizar en aceptar o rechazar en un número finito de pasos para toda entrada x .

b

Para demostra que $L_3(M_3) = L_1 \cap L_2$, suponemos lo siguiente.

Suponemos que L_1 es decidable si existe una maquina M_1 tal que $L(M_1) = L_1$ y M_1 es total. De la misma forma suponemos un L_2 que es decidable si existe una maquina M_2 tal que $L(M_2) = L_2$ y M_2 es total.

Por lo tanto lo que se quiere demostrar es que existe una M_3 tal que $L_3(M_3) = L_1 \cap L_2$ y M_3 es total.

Suponemos el funcionamiento de $M_3 = U$ como el siguiente:

1. M_3 simula a M_1 con la entrada x
2. Si $M_1(x) = 0$ rechaza, M_3 rechaza
3. Si $M_1(x) = 1$ acepta, entonces M_3 simula a M_2 con la entrada x
4. Si $M_2(x) = 1$ acepta, M_3 acepta
5. Si $M_2(x) = 0$ rechaza, M_3 rechaza

Si suponemos que $M_1(x) = 1$, entonces $L_1(M_1)$ es reconocible, de esta manera para la misma x , $M_2(x) = 1$ acepta; $L_1(M_1) \cap L_2(M_2)$ se cumple.

Si suponemos que $M_2(x) = 1$, entonces $L_2(M_2)$ es reconocible, entonces para la misma x , $M_1(x) = 1$ acepta, de igual manera es reconocible; $L_1(M_1) \cap L_2(M_2)$ se cumple.

Dado que M_1 es una máquina total, la primera simulación siempre finaliza en un número finito de pasos. Si M_1 rechaza, M_3 termina inmediatamente. Si M_1 acepta, se pasa a simular M_2 , la cual también es total y garantiza terminar en un número finito de pasos. En ningún caso M_3 entra en un bucle infinito, demostrando que M_3 es total.

Por lo tanto, $L(M_3) = L_1 \cap L_2$ y M_3 es total, lo que demuestra que la clase de lenguajes decidibles es cerrada bajo intersección.

C

Para demostrar que $L_3(M_3) = L_1 L_2$, suponemos lo siguiente.

Suponemos que L_1 es decidable si existe una maquina M_1 tal que $L(M_1) = L_1$ y M_1 es total. De la misma forma suponemos un L_2 que es decidable si existe una maquina M_2 tal que $L(M_2) = L_2$ y M_2 es total.

Por lo tanto lo que se quiere demostrar es que existe una M_3 tal que $L_3(M_3) = L_1L_2$ y M_3 es total. Para esto suponemos una maquina de turing tal que M_3 simule a M_1 y M_2 , así. Suponemos que $L_3 = \{x = yz : y \in L_1, z \in L_2\}$

1. Para cada posible división de la cadena x en dos partes y y z tales que $x = yz$ (desde $y = \epsilon, z = x$ hasta $y = x, z = \epsilon$):
 - (a) Simular M_1 con la entrada y
 - (b) Simular M_2 con la entrada z
 - (c) Si $M_1(y) = 1$ y $M_2(z) = 1$, entonces U
2. Si U prueba todas las posibles divisiones $x = yz$ y en ninguna se cumple para M_1 y M_2 ambas acepten, entonces M_3 rechaza.

Dada la cadena de entrada, se simula la primera parte de x en M_1 hasta alcanzar un estado final y, a continuación, se pasa a M_2 para completar la segunda mitad del cómputo.

Puesto que una cadena x de longitud n solo tiene un número finito de particiones las cuales son $n + 1$ posibles divisiones y M_1 y M_2 son máquinas totales que siempre se detienen, el bucle de M_3 siempre terminará en un número finito de pasos. Entonces la concatenación de los lenguajes con sus respectivas maquinas como $L_1(M_1)$ y $L_2(M_2)$; L_1L_2 es cerrado bajo la operación de concatenación.

d

Para demostrar que $L_3(M_3) = (L_1(M_1))^c$

Por definición, sabemos que una maquina es deidible si existe una M_3 tal que $L_3(M_3) = L_1(M_1)^c$. Así, suponemos el siguiente funcionamiento de la maquina $M_3 = U$.

1. U simula M_1 con x . Si $M_1(x) = 0$, entonces U acepta
2. U simula M_1 con x . Si $M_1(x) = 1$, entonces U rechaza
3. U rechaza en otro caso

Dado que M_1 es una máquina total, para cualquier entrada $x \in \Sigma^*$ solo existen dos posibilidades:

Si $x \in (L(M_1))^c$, entonces $x \notin L(M_1)$, lo que significa que $M_1(x) = 0$. Por el paso (1), M_3 acepta x . Si $x \notin (L(M_1))^c$, entonces $x \in L(M_1)$, lo que significa que $M_1(x) = 1$. Por el paso (2), M_3 rechaza x .

Dado que M_3 siempre se detiene e invierte exactamente las respuestas de M_1 , se cumple que $L(M_3) = (L(M_1))^c$ y M_3 total. Por lo tanto, el complemento de cualquier lenguaje decidable es también decidable.

e

¿Para cuáles de las operaciones anteriores se mantiene la cerradura para las clases P, NP?

Para la Clase P: Para decidir $L_1 \cup L_2$ o $L_1 \cap L_2$, basta con simular secuencialmente N_1 y N_2 sobre la entrada. La suma de tiempos polinómicos sigue siendo un polinomio, ya que como explicamos en la **union**, se ejecutan en tiempo polinomial, dado las M.T deterministas es lo que tardan en realizar su ejecución de transiciones de estados. El tiempo total es $O(n^{k_1} + n^{k_2}) = O(n^{\max(k_1, k_2)})$, al igual que la **intersección**, el cual sigue siendo polinomial.

Respecto a la **concatenación**, Para $L_1 L_2$, existen $n + 1$ formas de dividir la entrada $x = yz$. Evaluar $M_1(y)$ y $M_2(z)$ para cada división toma tiempo $O(n \cdot (n^{k_1} + n^{k_2}))$, el cual se mantiene polinomial. Finalmente para el **complemento**, dado que una M.T determinista para un lenguaje en P siempre se detiene y da una respuesta en tiempo polinómico, basta con intercambiar los estados de aceptación y rechazo de la máquina para decidir el complemento en tiempo polinómico. Si $L \in P$, existe una máquina determinista M de tiempo $O(n^k)$. Al ser determinista, basta con invertir las respuestas de aceptación y rechazo. La nueva máquina decide L^c en el mismo tiempo $O(n^k)$, demostrando que $L^c \in P$. [1]

Para la Clase NP:

Se mantienen en NP mediante computación no determinista. Esto quiere decir que para la unión y la intersección, una MTN ramifica de manera no determinista para simular M_1 o M_2 , es decir.

Para la **union**, se construye una MTN N_3 para procesar la entrada x de longitud $n = |x|$, se recibe la entrada x , N_3 realiza una ramificación no determinista en el primer paso:

Rama 1 simular N_1 con la entrada x .

Rama 2 simular N_2 con la entrada x .

Si la simulación en la rama elegida acepta, esa rama de N_3 acepta.

Existe al menos una ruta de cómputo aceptadora en $N_1(x)$ o en $N_2(x)$ si y solo si, existe al menos una ruta aceptadora en $N_3(x)$. La ramificación inicial toma $O(1)$ pasos. La profundidad máxima del árbol de cómputo es el máximo de las profundidades de N_1 y N_2 :

$$T_{N_3}(n) = O(1) + \max(T_{N_1}(n), T_{N_2}(n)) = O(n^{\max(k_1, k_2)})$$

Dado que la profundidad del árbol de cómputo no determinista está acotada por un polinomio, $L_1 \cup L_2 \in NP$.

Sucede algo similar para la **intersección** $L_1 \cap L_2 \in NP$. Se construye una MTN N_3 para procesar la entrada. N_3 simula de manera no determinista a N_1 sobre x que es la entrada.

Si alguna rama de $N_1(x)$ llega a un estado de aceptación, esa misma rama procede a simular de manera no determinista a N_2 sobre la entrada x .

Si para $N_2(x)$ en esa rama también alcanza un estado de aceptación, N_3 acepta.

Existen rutas válidas de aceptación tanto para $N_1(x)$ como para $N_2(x)$ si existe una secuencia de ramificaciones secuenciales en $N_3(x)$ que lleva a la aceptación. La profundidad máxima del árbol de cómputo resultante de ejecutar ambas simulaciones en secuencia y de igual manera es la suma de sus profundidades individuales, esto nos da

$$T_{N_3}(n) = T_{N_1}(n) + T_{N_2}(n) = O(n^{k_1}) + O(n^{k_2}) = O(n^{\max(k_1, k_2)})$$

Como la suma de dos polinomios es un polinomio, la profundidad total permanece acotada polinomialmente, demostrando que $L_1 \cap L_2 \in NP$.

Para la **concatenación** sucede de igual manera, proponiendo un N_3 a simula de manera no determinista a N_1 y N_2 sobre x que es la entrada, N_3 simula a N_1 sobre la subcadena y . Si la simulación de $N_1(y)$ no acepta en esa ruta, esa rama rechaza. Si acepta, procede a la siguiente etapa que es simular la segunda parte, N_3 simula a N_2 sobre la subcadena z . Si $N_2(z)$ también llega a un estado de aceptación en esa rama, N_3 acepta.

La fase de previsión o como adivinanza toma $O(n)$ pasos para escribir la partición y y z . La simulación secuencial de N_1 sobre y y N_2 sobre z toma profundidades máximas acotadas por sus polinomios, lo que nos da $T_{N_3}(n) \leq O(n + n^{k_1} + n^{k_2}) = O(n^{\max(k_1, k_2)})$.

Finalmente para el **complemento** se mantiene una incertidumbre, ya que si NP es cerrada bajo complemento, consistiría en intercambiar los estados finalizadores de una MTN, pero esta no decide el complemento, pues el criterio de aceptación no determinista exige que todas las ramas rechacen. Suponemos una N_3 MTN acepta una cadena de entrada w si al menos una de sus múltiples ramas de cómputo alcanza un estado de aceptación.

Para que una cadena w pertenezca al complemento L_3^c , todas las ramas de la MTN deben rechazar la entrada, por lo que si en una MTN N_3 intercambia los estados de aceptación y rechazo para formar N'_3 , la nueva máquina aceptará la cadena si al menos una rama alcanzaba el estado de rechazo original. Si una cadena $w \in L_3^c$ generaba algunas ramas de

aceptación y algunas de rechazo en N_3 , la máquina modificada N'_3 vería las ramas de rechazo originales convertidas en aceptación y también aceptaría w . Por lo tanto, N'_3 no reconoce el complemento.

El complemento de un lenguaje en NP pertenece a una clase conocida como $co-NP$, y además en el artículo [2] se conjetura que $NP \neq co-NP$. [2]

Ejercicio 4

Enunciado

Sea $\Sigma = \{s_1, s_2, \dots, s_k\}$, con $k \in \mathbb{N}$.

- Demuestra que Σ^* es contable.
- ¿El conjunto de máquinas de Turing NO-Deterministas es contable? considera la versión de MT NO determinista vista en clase.
- ¿Qué podemos concluir de las afirmaciones de los incisos anteriores?

Solución al Inciso a

Enunciado

Sea $\Sigma = \{s_1, s_2, \dots, s_k\}$, con $k \in \mathbb{N}$. Demuestra que la cerradura de Kleene Σ^* es un conjunto **contable** (o **numerable**).

Datos

- Alfabeto (Σ):** Es un conjunto finito de k símbolos, $\Sigma = \{s_1, s_2, \dots, s_k\}$.
- Conjunto de cadenas de longitud n (Σ^n):** Es el conjunto formado por todas las secuencias de exactamente n símbolos tomados de Σ . Como en cada una de las n posiciones hay k opciones posibles, la cardinalidad de este conjunto es estrictamente finita:

$$|\Sigma^n| = k^n$$

- Cerradura de Kleene (Σ^*):** Es el conjunto de todas las cadenas finitas que se pueden construir con los símbolos del alfabeto Σ , incluyendo la cadena vacía ϵ (donde $|\epsilon| = 0$):

$$\Sigma^* = \bigcup_{n=0}^{\infty} \Sigma^n = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$$

- Conjunto Contable:** Un conjunto A es contable si existe una **función biyectiva** $f : \mathbb{N} \rightarrow A$ (es decir, si es posible listar todos sus elementos en una secuencia ordenada sin omitir ni repetir ninguno).

P.D. que Σ^* es contable

Para demostrar que Σ^* es contable, construiremos una enumeración explícita asignándole a cada cadena $w \in \Sigma^*$ un índice entero único $i \in \mathbb{N}$ mediante el **orden lexicográfico por longitud**.

Regla de ordenamiento:

1. **Por longitud creciente:** Listamos primero la cadena de longitud 0, luego todas las cadenas de longitud 1, después las de longitud 2, y así sucesivamente.
2. **Por orden lexicográfico:** Para cadenas que comparten la misma longitud n , las ordenamos alfabéticamente fijando una jerarquía en los símbolos: $s_1 < s_2 < \dots < s_k$.

Enumeración

Aplicando esta regla, obtenemos la siguiente lista ordenada e infinita de elementos de Σ^* :

1. **Longitud $n = 0$ (Σ^0):** Contiene $k^0 = 1$ cadena:

$$w_1 = \epsilon$$

2. **Longitud $n = 1$ (Σ^1):** Contiene $k^1 = k$ cadenas:

$$w_2 = s_1, \quad w_3 = s_2, \quad \dots, \quad w_{k+1} = s_k$$

3. **Longitud $n = 2$ (Σ^2):** Contiene k^2 cadenas:

$$w_{k+2} = s_1s_1, \quad w_{k+3} = s_1s_2, \quad \dots, \quad w_{k+1+k^2} = s_k s_k$$

4. **Longitud $n = L$ (Σ^L):** Contiene k^L cadenas.

Dado que cada bloque de longitud Σ^n es **finito**, para cualquier cadena arbitraria $w \in \Sigma^*$ de longitud L , la cantidad de cadenas que la anteceden en la lista es finita y está dada por la suma de las potencias de k :

$$\text{Posición inicial de los elementos de } \Sigma^L = 1 + \sum_{n=0}^{L-1} k^n = 1 + \frac{k^L - 1}{k - 1} \quad (\text{para } k > 1)$$

Conclusión

- **Sobreyectividad:** Cualquier cadena posible $w \in \Sigma^*$ tiene una longitud finita L , por lo que ocupará una posición finita y bien definida $i \in \mathbb{N}$ en la lista.
- **Inyectividad:** Dos posiciones distintas $i \neq j$ corresponden a dos cadenas distintas $w_i \neq w_j$.

La asignación $f(i) = w_i$ establece una **biyección explícita entre $\mathbb{N}$ y Σ^*** , lo que prueba formalmente que Σ^* **es un conjunto contable**.

Solución al Inciso b

Enunciado

¿El conjunto de máquinas de Turing NO-Deterministas es contable? Considera la versión de MT NO determinista vista en clase.

Sí, el conjunto de todas las Máquinas de Turing No-Deterministas (MTND) es contable.

A pesar de que el no determinismo permite a la máquina ramificar sus cálculos y generar un árbol con múltiples rutas de decisión (árbol de cómputo), la estructura de cualquier MTND es estrictamente finita. Por lo tanto, el conjunto de todas las MTNDs posibles tiene cardinalidad numerable.

P.D. que el conjunto de máquinas de Turing NO-Deterministas es contable

Definición de la MTND

Una Máquina de Turing No-Determinista se define formalmente como una 7-tupla:

$$N = \langle Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r \rangle$$

Donde:

1. $Q = \{q_0, q_1, \dots, q_{m-1}\}$ es un conjunto **finito** de m estados.
2. $\Sigma = \{s_1, \dots, s_k\}$ es el alfabeto **finito** de entrada.
3. $\Gamma = \{s_1, \dots, s_k, \sqcup, \dots, s_p\}$ es el alfabeto **finito** de la cinta.
4. $q_0 \in Q$ es el estado inicial, $q_a \in Q$ es el estado de aceptación y $q_r \in Q$ es el estado de rechazo.
5. δ es la **función de transición no-determinista**:

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{\leftarrow, \rightarrow, -\})$$

o equivalentemente $\delta \subseteq (Q \times \Gamma) \times (Q \times \Gamma \times \{\leftarrow, \rightarrow, -\})$. Para cada par (q, s) , la máquina tiene un número **finito** de posibles transiciones.

Codificación de una MTND como una cadena binaria finita

Dado que los conjuntos Q , Σ , Γ son finitos y la función de transición δ contiene un número finito de pares y opciones de transición, **todos los elementos que conforman una MTND N se puede codificar como una cadena binaria de longitud finita $\langle N \rangle \in \{0, 1\}^*$** :

- **Codificación de Estados:** Los estados $q_0, q_1, \dots, q_{m-1}$ se representan con cadenas de ceros: $q_i \Rightarrow 0^{i+1}$.

- **Codificación del Alfabeto:** Los símbolos de Γ se representan análogamente: $s_j \Rightarrow 0^{j+1}$ (reservando una representación específica para el símbolo en blanco $\sqcup$).
- **Codificación de Movimientos:** Las direcciones $\{\leftarrow, \rightarrow, -\}$ se representan como: $- \Rightarrow 0$, $\leftarrow \Rightarrow 00$, $\rightarrow \Rightarrow 000$.
- **Codificación de las Transiciones:** Cada regla no-determinista de la forma $((q_i, s_j), (q_{i'}, s_{j'}, m))$ se codifica como un bloque delimitado por el símbolo 1:

$$0^{i+1} 1 0^{j+1} 1 0^{i'+1} 1 0^{j'+1} 1 0^{\text{movimiento}}$$

- **MTND Completa:** Una máquina no determinista N que tenga múltiples alternativas de transición para el mismo par de estado y símbolo simplemente lista todas sus reglas separadas por separadores dobles (11).

Dado que la descripción de N consta de un número finito de estados y reglas, la cadena codificada resultante $\langle N \rangle$ tiene una **longitud finita** $|\langle N \rangle|$.

Comprobación de que el conjunto de MTND es contable

Definimos la función f que asigna a cada Máquina de Turing No Determinista con su respectiva cadena binaria única que lo representa:

$$f : \mathcal{M}_{\text{ND}} \rightarrow \{0, 1\}^* \quad \text{donde} \quad f(N) = \langle N \rangle$$

- **Sobreyectividad (nota):** Creemos importante mencionar que la función de codificación f no es sobreyectiva sobre todo el conjunto $\{0, 1\}^*$ e incluso no es necesario que lo sea para demostrar que el conjunto es contable. Basta con que la función sea inyectiva, pues no es necesario construir una biyección directa contra todo el codominio.

De manera general, si existe una inyección $f : A \rightarrow B$, la cardinalidad de A es menor o igual a la de B ($|A| \leq |B|$). Como f es inyectiva, induce una **biyección** entre A y su **imagen** $\text{Im}(f) = f(A) \subseteq B$.

- **Inyectividad:** Dos máquinas no deterministas distintas $N_1 \neq N_2$ difieren en al menos un estado, símbolo o regla de transición no determinista. Por lo tanto, sus representaciones binarias son distintas: $\langle N_1 \rangle \neq \langle N_2 \rangle$.

Esto establece que el conjunto de todas las MTNDs ($\mathcal{M}_{\text{ND}}$) es biyectivo con un subconjunto de todas las cadenas binarias $\{0, 1\}^*$. Cabe señalar que f no sea biyectiva con todas las cadenas posibles de $\{0, 1\}^*$, porque no cualquier cadena binaria resulta ser una máquina válida (por ejemplo 11111).

Conclusión

En el **inciso (a)** demostramos que para cualquier alfabeto finito Σ , la cerradura de Kleene Σ^* (y en particular $\{0, 1\}^*$) es un conjunto **contable (numerable)**. Además, todo subconjunto de un conjunto contable es a su vez **contable o finito**, y como $\mathcal{M}_{\text{ND}}$ equivale a un subconjunto de $\{0, 1\}^*$, concluimos que **el conjunto de todas las Máquinas de Turing No Deterministas es contable**.

Solución al Inciso c

Enunciado

¿Qué podemos concluir de las afirmaciones de los incisos anteriores?

De la combinación de los incisos **(a)** y **(b)** —junto con la teoría de complejidad y computabilidad vista en el curso— consideramos las siguientes ideas:

1. Igualdad de Cardinalidad entre Algoritmos Deterministas y No Deterministas

De acuerdo al **inciso (a)** demostramos que el conjunto de cadenas finitas Σ^* (y por lo tanto $\{0, 1\}^*$) es **contable**, lo que implica que el conjunto de todas las Máquinas de Turing Deterministas (MTD) es contable; siendo su demostración prácticamente análoga a la que hicimos para las MTND, lo único que cambia es la descripción de la función de transición y por ende su codificación, pero manteniendo el número finito de posibles transiciones.

En el **inciso (b)** se demostró que el conjunto de todas las Máquinas de Turing No Deterministas (MTND) también es **contable**.

Por lo tanto, ambos modelos tienen exactamente la misma cantidad de elementos:

$$|\mathcal{M}_D| = |\mathcal{M}_{ND}|$$

El no determinismo permite a una máquina ramificar sus cálculos en un árbol de decisiones, pero **no incrementa la cantidad de algoritmos que se pueden especificar en el conjunto universo**. La descripción de cualquier máquina (determinista o no determinista) sigue siendo una cadena finita de símbolos.

2. Existencia Absoluta de Problemas Indecidibles (Principio del Palomar)

Un **problema de decisión** sobre un alfabeto Σ es una función $f : \Sigma^* \rightarrow \{0, 1\}$, la cual equivale a seleccionar un lenguaje $L \subseteq \Sigma^*$.

El conjunto de todos los problemas de decisión posibles corresponde al conjunto potencia $\mathcal{P}(\Sigma^*)$.

Dado que Σ^* es un conjunto contable (demostrado en el inciso a), su conjunto potencia $\mathcal{P}(\Sigma^*)$ es **incontable** por el Teorema de Cantor y la prueba de diagonalización.

Tenemos por el Principio del Palomar:

- **Palomares (Algoritmos/Máquinas):** Cantidad contable.
- **Palomas (Problemas de decisión):** Cantidad incontable.

Como hay estrictamente más problemas que Máquinas de Turing (tanto deterministas como no deterministas), se concluye que **existen infinitos problemas que no pueden ser resueltos por ningún algoritmo posible (problemas indecidibles)**.

3. El No Determinismo NO Supera la Frontera de la Computabilidad

Aunque el no determinismo es una herramienta teórica poderosa para definir clases de complejidad como **NP**, **no amplía el conjunto de problemas que son computables en sentido absoluto.**

Una MTND no puede decidir problemas que sean fundamentalmente indecibles para una MTD (como el *Problema del Paro*).

Esto reafirma la **Tesis de Church-Turing**: el conjunto de lenguajes decidibles/reconocibles por cualquier modelo razonable de cómputo (ya sean MTs de una cinta, k -cintas, RAMs o MTNDs) coincide exactamente con la clase de lenguajes procesables por la Máquina de Turing estándar.

Ejercicio 5

Enunciado

Investiga ejemplos de problemas no computables. Ejemplifica los problemas y justifica por qué no son computables.

Solución

Un **problema de decisión** (formalizado como un lenguaje $L \subseteq \Sigma^*$ o una función $f : \Sigma^* \rightarrow \{0, 1\}$) es **no computable** (o **indecidible**) si **no existe ninguna Máquina de Turing M capaz de resolverlo y detenerse para todas las entradas posibles.**

Si un problema es indecidible, significa que ningún algoritmo podrá jamás dar una respuesta correcta de forma garantizada para cualquier ejemplar del problema en un tiempo finito y eficiente (en tiempo polinomial).

A continuación, presentamos el análisis correspondiente para 2 problemas no computables:

Ejemplo 1: El Problema de Correspondencia de Post (*Post Correspondence Problem*, PCP [4]) [5]

Definición: Dado un conjunto finito de “dominós” o pares de cadenas $P = \left\{ \left[\begin{smallmatrix} u_1 \\ v_1 \end{smallmatrix} \right], \left[\begin{smallmatrix} u_2 \\ v_2 \end{smallmatrix} \right], \dots, \left[\begin{smallmatrix} u_k \\ v_k \end{smallmatrix} \right] \right\}$ sobre algún alfabeto, determinar si existe una secuencia de índices $i_1, i_2, \dots, i_m$ (con posibles repeticiones) tal que la concatenación de las cadenas superiores sea exactamente igual a la concatenación de las cadenas inferiores:

$$u_{i_1} u_{i_2} \dots u_{i_m} = v_{i_1} v_{i_2} \dots v_{i_m}$$

Ejemplar:

Considera las siguientes 3 fichas:

$$F_1 = \left[\frac{a}{b} \right], \quad F_2 = \left[\frac{a}{aa} \right], \quad F_3 = \left[\frac{aab}{a} \right]$$

- ¿Existe una coincidencia? **Sí**. La secuencia de fichas (F_2, F_3, F_1, F_2) da:
 - **Cadena Superior:** $a + aab + a + a = \mathbf{aaabaa}$
 - **Cadena Inferior:** $aa + a + b + aa = \mathbf{aaabaa}$
- El problema general del PCP exige determinar si **cualquier** conjunto arbitrario de fichas dado tiene o no una solución.

El PCP se demuestra indecidible mediante una **reducción desde el Problema de Aceptación de la MT**. Se demuestra que cualquier configuración de cómputo de una Máquina de Turing (la secuencia de configuraciones válidas $C_1 \vdash C_2 \vdash \dots \vdash C_k$) se puede codificar como una colección de fichas del PCP.

Una coincidencia en el PCP existe **si y sólo si** la Máquina de Turing alcanza un estado de aceptación. Dado que determinar si una MT acepta una entrada es indecidible, el **PCP es no computable**.

Ejemplo 2: El Problema del Lenguaje Vacío de una MT y Teorema de Rice [5]

Definición: Dada la descripción de una Máquina de Turing M , determinar si el lenguaje que reconoce es completamente vacío:

$$E_{TM} = \{ \langle M \rangle \mid L(M) = \emptyset \}$$

Ejemplar: Preguntarse si existe al menos una entrada w para la cual la máquina M responda “Aceptar”, o si por el contrario no acepta absolutamente ninguna cadena del universo.

El **Teorema de Rice** establece que *cualquier propiedad semántica no trivial de los lenguajes reconocidos por Máquinas de Turing es indecidible*.

1. Ser un lenguaje vacío ($L(M) = \emptyset$) es una **propiedad semántica** (depende del lenguaje aceptado por la máquina, no de su sintaxis).
2. Es **no trivial** porque algunas MTs reconocen el lenguaje vacío y otras no.

Por el Teorema de Rice, **El Problema del Lenguaje Vacío de una MT es automáticamente no computable**.

Referencias

- [1] K. R. Chowdhary. *Theory of Computation: Automata, Formal Languages, Computation and Complexity*. Springer Nature, 2025. Disponible en: <https://doi.org/10.1007/978-981-97-6234-7>.

-
- [2] DeJesse, N., Lyudovik, S., Pai, D., & Reidy, M. (2025). *A critique of Lin's "On NP versus coNP and Frege Systems"*. arXiv. <https://arxiv.org/abs/2505.05658>
 - [3] M. Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3.^a ed., 2013.
 - [4] Post correspondence problem. (2026). In *Wikipedia*. https://en.wikipedia.org/w/index.php?title=Post_correspondence_problem&oldid=1361182568
 - [5] Carrión Viramontes, J. (n.d.). *Teoría de la Computación*. Retrieved October 1, 2026, from <https://dsc.itmorelia.edu.mx/~jcolivares/nlp/libros/teoriaComputacion.pdf>